Unit 5: Pointer & Virtual Functions

Prof. Mahipal Khoja
Assistant Professor
Artificial Intelligence and Data Science Department

Content

1. Pointers to objects
2. Pointer to Derived Classes
3. Virtual Functions
4. Pointer to Virtual Functions
5. ‘this’ Pointer

Pointer:

A pointer is a variable that stores the memory address of another variable or object. Instead of holding a value directly, it points to the location where the value is stored in memory.

Why pointers are important in C++

- Access memory directly
- Pass arguments efficiently to functions
- Create dynamic memory using new and delete
- Implement data structures (arrays, linked lists, trees)
- Support **polymorphism** using base-class pointers

Declaring a Pointer

data_type *pointer_name;

Ex:- int *p;

Operators

Operator	Purpose
&	Address-of operator
*	Dereference operator

Basic Pointer Example

```
#include <iostream>
using namespace std;

int main() {
    int a = 5;        // Normal variable
    int *p;           // Pointer declaration
    p = &a;           // Store address of a

    cout << "Value of a: " << a << endl;
    cout << "Address of a: " << &a << endl;
    cout << "Pointer p stores: " << p << endl;
    cout << "Value using pointer: " << *p << endl;

    return 0;
}
```

Output explanation:

1 Value of a: 5
a is assigned value 5, so it prints 5.
2 Address of a: 0x7ffeefbff5ac
&a prints the memory address of variable a.
This address is system-dependent.
3 Pointer p stores: 0x7ffeefbff5ac
p = &a, so pointer p stores the same address as a.
4 Value using pointer: 5
*p dereferences the pointer.
It accesses the value stored at the address → 5.

Types of Pointers

1. Null pointer

Points to nothing. Used to avoid garbage values.

```
int *p = NULL;
```

2. Void pointer

Can point to any data type but must be typecast before use.

```
void *p;
```

3. Pointer to pointer

Stores address of another pointer.

```
int a = 10;
int *p = &a;
int **pp = &p;
```

Types of Pointers

4. Pointer to Array
   Stores the **address of the first element of an array**
   int arr[3] = {1, 2, 3};
   int *p = arr;

5. **Pointer to object**

Pointer to Object

A pointer to object is a pointer that stores the address of an object of a class.

- Just like int* points to an integer
- ClassName* points to an object of that class

Syntax

ClassName *ptr;

- ClassName → type of object
- *ptr → pointer variable
- ptr will store the **address of an object**

Pointer to Object

```
#include <iostream>
using namespace std;

class Student {
public:
    int roll;
    void display() {
        cout << "Roll number: " << roll << endl;
    }
};
int main() {
    Student s;        // object
    Student *p;       // pointer to object
    p = &s;           // pointer stores address of object
    p->roll = 1;      // access data member
    p->display();     // call member function

    return 0;
}
```

Pointer to Object

Step-by-step Explanation

Student s;
Creates an object in memory
Student *p;
Declares a pointer that can point to a Student object
p = &s;
Pointer stores the address of object s
p->roll = 1;
Accesses roll using **arrow operator**
p->display();
Calls object's member function using pointer

Pointer to Object

Arrow Operator

"When we use an object directly, we use the dot (.) operator. When we use a pointer to an object, we use the arrow (->) operator."

Comparison

- s.roll = 1; // object
- p->roll = 1; // pointer to object

Pointer to Derived Class

If a derived class is created from a base class, then logically the derived object **is also a base object**.

So the question is ….

Can a base class pointer point to a derived class object?

Pointer to Derived Class

A pointer to derived class means:
A base class pointer pointing to an object of a derived class.

Note:-

The pointer is still a base class pointer
Only the object is of derived class

Syntax:-

```
Base Class *ptr;
Derived Class obj;
ptr = &obj;
```

Pointer to Derived Class

Example

```
#include <iostream>
using namespace std;

class Base {
public:
    void show() {
        cout << "This is Base class show()" << endl;
    }
};
class Derived : public Base {
public:
    void show() {
        cout << "This is Derived class show()" << endl;
    }
};
```

Pointer to Derived Class

```
int main() {
    Base *b;
    Derived d;
    b = &d;     // Base pointer points to Derived object
    b->show();  // Function call
    return 0;
}
```

Step-by-Step Explanation

class Base
→ contains a function show()
class Derived : public Base
→ inherits Base and redefines show()
Base *b;
→ base class pointer

Derived d;
→ derived class object
b = &d;
→ base pointer now points to derived object
b->show();
→ **Base class function is called**

Pointer to Derived Class

Note:

Pointer type decides which members are accessible
Base pointer can access only base class members
Derived-specific members are not accessible through base pointer

Example:-

b->derivedFunction(); // ❌ Not allowed

Pointer to Derived Class

Even though the pointer points to a derived object, the base version of the function is called.

What if we want the derived version to execute?

That is where **virtual functions** come in.

Virtual Functions

A virtual function is a member function of a class that is declared in the base class and is meant to be overridden in the derived class, so that the function call is decided at runtime, not at compile time.

Syntax

```
virtual return_type function_name();
```

Why Virtual Functions Are Needed

- Pointer type = Base
- Object type = Derived
- We want behavior of Derived, not Base
- Virtual functions make C++ look at the object, not the pointer

Virtual Functions

Example

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual void show() {
        cout << "This is Base class show()" << endl;
    }
};
```

Virtual Functions

```
class Derived : public Base {
public:
   void show() {
        cout << "This is Derived class show()" << endl;
    }
};
int main() {
    Base *b;
    Derived d;
   b = &d;      // Base pointer → Derived object
    b->show();   // Now calls Derived version
    return 0;
}
```

Pointer to Virtual Functions

When a base class pointer points to a derived class object and calls a virtual function, the compiler uses a Virtual Table (vtable) to determine which function to execute.

Working Mechanism
Base class maintains a vtable
Derived class updates vtable entry
Pointer accesses vtable at runtime
Correct function is executed

Advantage

- Enables true runtime polymorphism
- Improves program flexibility

Pointer to Virtual Functions

The term "pointer to virtual function" does not represent a new or separate type of pointer in C++. Instead, it describes a situation where a virtual function is called using a base class pointer.

In this case:

The pointer points to an object (not a function)
The function being called is virtual
The decision of which function to execute is made at runtime

This Pointer

The **this pointer** is an implicit pointer available inside all non-static member functions of a class. It holds the address of the current calling object.

In simple words:
this points to the object that is currently using the member function.

Why the this Pointer Is Needed

- Data members and function parameters have the same name
- An object needs to pass itself to another function
- A function needs to return the calling object

This Pointer

```
#include <iostream>
using namespace std;
class Sample {
int x;
public:
void set(int x) {
this->x = x; // Distinguishes data member from parameter
}
void show() {
cout << "Value of x: " << x << endl;
}
};
int main() {
Sample obj;
obj.set(10);
obj.show();
return 0;
}
```

This Pointer

Explanation of the Program
int x; is a data member of the class
void set(int x) has a parameter with the same name

Inside set():

- x refers to the parameter
- this-> x refers to the object's data member

The value is stored correctly in the calling object.

Parul® University

NAAC GRADE A++

https://paruluniversity.ac.in/